

UNIVERSIDAD DE CÓRDOBA

Práctica Final de Sistemas Interactivos 2025-26

*Diseño e implementación de una interfaz de gestión para
peluquería*

Autor: Alicia Muriel Fernández

Asignatura: Sistemas Interactivos

Curso: 2025/26

Entrega: Práctica Final

27 de abril de 2026

Índice general

Resumen	2
1. Introducción al problema	3
1.1. Descripción del problema	3
1.2. Objetivo de la interfaz	3
1.3. Plataforma y público objetivo	4
1.4. Consideraciones de diseño	4
1.5. Requisitos mínimos de la interfaz	4
2. Antecedentes	5
3. Diseño centrado en el usuario	6
3.1. Usuarios tipo	6
3.1.1. Profesional	6
3.1.2. Cliente	6
3.2. Escenarios de uso	6
3.3. Wireframes	7
4. Implementación de la interfaz	8
4.1. Organización del código	8
4.2. Diagrama de clases	8

4.3. Aspectos significativos de la implementación	8
4.3.1. Internacionalización	8
4.3.2. Gestión de datos	8
4.3.3. Decisiones de diseño	10
4.3.4. Aplicación de conceptos teóricos	10
5. Tutorial / breve manual de usuario	11
5.1. Compilación	11
5.2. Ejecución	11
5.3. Descripción detallada de las pantallas	11
5.3.1. 1. Pantalla de Inicio de Sesión (Login)	11
5.3.2. 2. Pantalla de Registro	12
5.3.3. 3. Pantalla de Recuperación de Contraseña	12
5.3.4. 4. Menú Principal del Cliente	13
5.3.5. 5. Pantalla de Reserva de Cita (Cliente)	13
5.3.6. 6. Pantalla de Anulación de Cita (Cliente)	13
5.3.7. 7. Panel de Administración Principal	14
5.3.8. 8. Panel de Inicio (Admin)	14
5.3.9. 9. Panel de Agenda (Admin)	15
5.3.10. 10. Diálogo Añadir Cita (Admin)	15
5.3.11. 11. Panel de Clientes (Admin)	15
5.3.12. 12. Ficha de Cliente (Admin)	16
5.3.13. 13. Panel de Inventario (Admin)	16
5.4. Gestores y Lógica de Negocio	16
5.4.1. GestorIdiomas	16

5.4.2. GestorUsuarios	17
5.4.3. GestorCitas	17
5.5. Diálogos Modales	17
5.5.1. DialogoErrorLogin	17
5.5.2. DialogoAnadirCita	18
5.6. Capturas de pantalla	18
6. Diferencias respecto al prototipo	20
6.1. Cambios de diseño justificados	20
6.1.1. Eliminación de iconografía visual (emojis)	20
6.1.2. Estructura de datos simplificada	20
6.1.3. Validaciones más estrictas	21
6.1.4. Gestión de pantallas revisada	21
6.1.5. Internacionalización completa	21
6.1.6. Componentes más modulares	22
6.2. Elementos conservados del prototipo	22
7. Conclusiones y Reflexión	23
7.1. Logros alcanzados	23
7.2. Aspectos mejorables	23
7.3. Conclusión final	24
A. Anexos	25
A.1. Comandos útiles	25

Resumen

Este documento describe la implementación de la interfaz gráfica desarrollada en Java Swing para la práctica final de Sistemas Interactivos. Se basa en el prototipo anterior (i32camol_i32mu fea_prototipo) y documenta las diferencias relevantes, la organización del código, los as

Capítulo 1

Introducción al problema

1.1. Descripción del problema

En muchas peluquerías y salones de belleza se sigue trabajando con agendas de papel, hojas sueltas y anotaciones manuales. Esta forma de gestión provoca pérdidas de citas, dificultades para consultar el historial del cliente, errores en la planificación de servicios y descontrol en el inventario.

Los principales problemas detectados son:

- Dificultad para mantener una agenda actualizada y evitar duplicidades.
- Falta de información centralizada del historial de cada cliente.
- Control manual del stock que genera carencias o exceso de producto.
- Proceso de reserva de cita poco accesible para el cliente.

1.2. Objetivo de la interfaz

El objetivo es ofrecer una interfaz que permita a los profesionales y a los clientes gestionar los procesos de reserva y administración de forma eficiente, clara y accesible.

- Para el profesional: gestionará agenda, clientes e inventario.
- Para el cliente: podrá reservar cita y consultar información básica sin intervención telefónica.

1.3. Plataforma y público objetivo

La aplicación está diseñada para escritorio con apariencia de interfaz móvil dentro de Java Swing, adaptada a dos perfiles principales:

- Profesional de peluquería: necesita herramientas para controlar citas, clientes y stock.
- Cliente: desea reservar cita de forma rápida y sin errores.

1.4. Consideraciones de diseño

Las decisiones de diseño se han basado en principios de usabilidad:

- Interfaz simple y coherente en todas las vistas.
- Feedback claro al usuario en acciones como registro, reserva y errores.
- Minimización de errores mediante entradas guiadas (listas desplegables, validación de teléfono, etc.).
- Uso de colores y botones contrastados para resaltar acciones importantes.

1.5. Requisitos mínimos de la interfaz

Según el guion de la práctica, la interfaz implementada debe incluir al menos:

- Pantalla principal para el profesional y para el cliente.
- Agenda con creación y cancelación de citas.
- Gestión de clientes con ficha técnica.
- Control de inventario.
- Internacionalización en, como mínimo, dos idiomas.

Capítulo 2

Antecedentes

La práctica se ha realizado como continuación del prototipo previo entregado en la primera fase. El prototipo original definió las pantallas y flujos básicos, sobre los que se ha construido la implementación actual en Java Swing.

Capítulo 3

Diseño centrado en el usuario

3.1. Usuarios tipo

3.1.1. Profesional

Usuario encargado de gestionar la agenda, consultar clientes y revisar inventario.

3.1.2. Cliente

Usuario final que desea reservar cita y consultar su información.

3.2. Escenarios de uso

Se contemplan dos flujos principales:

- Reserva de cita por cliente.
- Gestión de agenda, clientes e inventario por el profesional.

3.3. Wireframes

Los wireframes iniciales del prototipo definían las pantallas principales de acceso, registro, agenda, ficha de cliente e inventario. La implementación final respeta la organización general, con algunas adaptaciones justificadas en el documento.

Capítulo 4

Implementación de la interfaz

4.1. Organización del código

El proyecto utiliza la siguiente estructura de directorios:

4.2. Diagrama de clases

A continuación se muestra un diagrama simplificado de las clases principales:

4.3. Aspectos significativos de la implementación

4.3.1. Internacionalización

La aplicación está internacionalizada en tres idiomas: español, inglés y francés. Se emplea la clase `GestorIdiomas` junto a los ficheros de recursos en `src/bundles/`:

- `TextoseS.properties` `TextosenGB.properties` `TextosfrFR.properties`

4.3.2. Gestión de datos

Los datos se guardan en archivos de texto plano:

```
src/  
  DialogoAnadirCita.java  
  DialogoErrorLogin.java  
  GestorCitas.java  
  GestorIdiomas.java  
  GestorUsuarios.java  
  PanelAdminAgenda.java  
  PanelAdminClientes.java  
  PanelAdminFichaCliente.java  
  PanelAdminInicio.java  
  PanelAdminInventario.java  
  PanelAdminPrincipal.java  
  PanelClienteAnular.java  
  PanelClienteDatos.java  
  PanelClienteMenu.java  
  PanelLogin.java  
  PanelRecuperar.java  
  PanelRegistro.java  
  PanelSolicitudAdmin.java  
  VentanaPrincipal.java  
bundles/  
  Textos_en_GB.properties  
  Textos_es_ES.properties  
  Textos_fr_FR.properties  
images/  
  (iconos y recursos gráficos)
```

Figura 4.1: Estructura de directorios del proyecto

Clases principales del proyecto:

- **VentanaPrincipal**: controlador principal que gestiona el cambio de vistas.
- **PanelLogin**, **PanelRegistro**, **PanelRecuperar**: pantallas de autenticación.
- **PanelAdminPrincipal**: navegación del administrador.
- **PanelAdminAgenda**, **PanelAdminClientes**, **PanelAdminInventario**: paneles de gestión.
- **PanelClienteMenu**, **PanelClienteDatos**, **PanelClienteAnular**: interfaz del cliente.
- **GestorIdiomas**, **GestorUsuarios**, **GestorCitas**: clases de lógica de negocio.
- **DialogoAnadirCita**, **DialogoErrorLogin**: diálogos modales.

Figura 4.2: Componentes principales de la aplicación

- `usuarios.txt` para el registro de usuarios.
- `citas.txt` para las reservas.
- `fichas_clientes.txt` para los datos de clientes.

4.3.3. Decisiones de diseño

Se ha utilizado Java Swing con paneles modulares para que cada pantalla sea independiente:

- `VentanaPrincipal` controla el cambio de vistas.
- `PanelAdminPrincipal` organiza la navegación entre paneles administrativos.
- `PanelClienteMenu` ofrece las opciones específicas del cliente.

4.3.4. Aplicación de conceptos teóricos

Se aplican conceptos vistos en clase como:

- Uso de `BorderLayout`, `GridBagLayout`, `GridLayout` y `FlowLayout`.
- Manejo de eventos con `ActionListener` y lambdas.
- Validación de formularios con retroalimentación visual.
- Separación de lógica de interfaz y acceso a datos mediante clases auxiliares.

Capítulo 5

Tutorial / breve manual de usuario

5.1. Compilación

Para compilar el proyecto desde la consola, situarse en la raíz del proyecto y ejecutar:

```
javac -d bin src/*.java
```

Si se utiliza un IDE, importar el proyecto como proyecto Java y compilar con el compilador estándar.

5.2. Ejecución

Para ejecutar la aplicación:

```
java -cp bin VentanaPrincipal
```

5.3. Descripción detallada de las pantallas

5.3.1. 1. Pantalla de Inicio de Sesión (Login)

La pantalla inicial de la aplicación presenta un formulario de autenticación bimodal. En la esquina superior derecha existe un botón para cambiar el idioma de la interfaz. El

formulario contiene dos campos de texto: uno para el teléfono (identificador de usuario) y otro para la contraseña. Existen tres botones principales:

- **Iniciar sesión:** valida las credenciales contra la base de datos de usuarios en `usuarios.txt`. Si son correctas y el usuario es administrador, redirige a `PanelAdminPrincipal`. Si es cliente, va a `PanelClienteMenu`.
- **Registro:** permite a nuevos usuarios crear una cuenta. Abre la pantalla de registro.
- **Recuperar contraseña:** permite buscar una cuenta existente por teléfono e intentar recuperar acceso.

Implementación: `PanelLogin.java` utiliza `BorderLayout` para organizar el formulario en el centro y botones en la base. La validación se realiza mediante `GestorUsuarios.obtenerRol()`.

5.3.2. 2. Pantalla de Registro

Pantalla para registrar nuevos usuarios con validación de datos:

- **Campo de teléfono:** valida exactamente 9 dígitos mediante expresión regular `d{9}`.
- **Campo de contraseña:** requiere mínimo 8 caracteres.
- **Confirmación de contraseña:** debe coincidir con el campo anterior.

Al registrarse, se guarda la línea `teléfono:contraseña:cliente` en `usuarios.txt`. Incluye validación para evitar duplicados. Si el registro es exitoso, muestra un diálogo de confirmación y regresa a login. Implementación: `PanelRegistro.java` con `GridBagLayout` para formularios.

5.3.3. 3. Pantalla de Recuperación de Contraseña

Permite a usuarios olvidados recuperar acceso:

- Campo de entrada de teléfono.
- Botón para buscar el usuario por teléfono.

- Si se encuentra, muestra un diálogo con la contraseña.
- Si no existe, muestra mensaje de error.

Implementación: `PanelRecuperar.java` con búsqueda lineal en `usuarios.txt`.

5.3.4. 4. Menú Principal del Cliente

Pantalla inicial tras autenticarse como cliente. Contiene dos botones principales en un panel con `GridLayout`:

- **Pedir cita:** abre `PanelClienteDatos` para reservar una nueva cita.
- **Anular cita:** abre `PanelClienteAnular` para visualizar y cancelar citas existentes.

Incluye opción de cierre de sesión para volver a login. Implementación: `PanelClienteMenu.java`.

5.3.5. 5. Pantalla de Reserva de Cita (Cliente)

Formulario para que el cliente seleccione fecha, hora y tipo de servicio:

- `JComboBox` para seleccionar día (listado dinámico o fijo).
- `JComboBox` para seleccionar hora.
- `JComboBox` para seleccionar el tipo de servicio.
- Botón de confirmación que guarda en `citas.txt`.
- Botón de cancelación que regresa al menú.

Implementación: `PanelClienteDatos.java` utiliza `GridBagLayout`. La reserva se almacena con formato `hora - nombreCliente`.

5.3.6. 6. Pantalla de Anulación de Cita (Cliente)

Muestra al cliente únicamente sus propias citas:

- **JList** con las citas del usuario actual filtradas de `citas.txt`.
- Botón **Anular cita** que elimina la cita seleccionada.
- Botón de retorno al menú.

Implementación: `PanelClienteAnular.java`.

5.3.7. 7. Panel de Administración Principal

Pantalla de navegación del administrador. En la parte superior hay botones de navegación con `GridLayout`:

- **Inicio**: muestra un resumen de citas del día.
- **Agenda**: gestión completa de citas.
- **Clientes**: visualización de clientes en grid.
- **Inventario**: gestión de productos.

En el área central (`pnlContenidoCentral`) se cambia dinámicamente el contenido según el botón pulsado. Implementación: `PanelAdminPrincipal.java` con `CardLayout` virtual mediante lógica de cambio de paneles.

5.3.8. 8. Panel de Inicio (Admin)

Resumen ejecutivo del administrador:

- Título: “Citas de hoy”
- Listado de citas programadas (ej: “10:00 - Ana García”, “11:30 - Juan Pérez”).
- Botón “Modificar agenda” que navega a la pantalla de agenda.

Implementación: `PanelAdminInicio.java` con `JList` en `BorderLayout`.

5.3.9. 9. Panel de Agenda (Admin)

Gestión completa de citas por el administrador:

- `DefaultListModel` con todas las citas del sistema desde `citas.txt`.
- Botón **Añadir cita**: abre `DialogoAnadirCita`.
- Botón **Anular cita**: elimina la cita seleccionada.

Implementación: `PanelAdminAgenda.java`. La cita se elimina reescribiendo `citas.txt` sin la línea seleccionada.

5.3.10. 10. Diálogo Añadir Cita (Admin)

Modal para añadir citas rápidamente:

- Selección de día/mes/año mediante `JComboBox`.
- Selección de hora mediante `JSpinner` o `JComboBox`.
- Campo de texto para el nombre del cliente.
- Botones Aceptar/Cancelar.

Implementación: `DialogoAnadirCita.java` con `GridBagLayout`.

5.3.11. 11. Panel de Clientes (Admin)

Visualización en grid de todos los clientes registrados:

- `GridLayout(0, 3)` para mostrar clientes en columnas de 3.
- Cada celda es un `JButton` con el nombre del cliente.
- Al hacer clic en un cliente, se abre `PanelAdminFichaCliente`.

Implementación: `PanelAdminClientes.java`.

5.3.12. 12. Ficha de Cliente (Admin)

Formulario para editar información del cliente seleccionado:

- `TextField` con el nombre (no editable).
 - `TextArea` para alergias (editable, multilinea).
 - `TextArea` para historial de servicios (editable, multilinea).
 - `TextField` para teléfono de contacto.
- Botón **Guardar**: persiste en `fichas_clientes.txt`. Botón **Cancelar**: *regresa a guardar*.
Implementación: `PanelAdminFichaCliente.java` con `GridBagLayout`.

5.3.13. 13. Panel de Inventario (Admin)

Gestión de productos y stock:

- Visualización en grid de productos (ej: “Champú”, “Tinte”, “Acondicionador”).
- Cada producto muestra su cantidad actual de stock.
- Botones para aumentar/disminuir stock.
- Botón de eliminación de productos.

Implementación: `PanelAdminInventario.java` con clase interna `Producto`.

5.4. Gestores y Lógica de Negocio

5.4.1. GestorIdiomas

Clase responsable de la internacionalización:

- Mantiene el idioma actual (`localeActual`) con tres opciones: `ES_ES`, `EN_GB`, `FR_FR`.
- Método `cambiarIdiomaBase()`: cicla entre idiomas en orden `ES → EN → FR → ES`.

- Método `getTexto(String clave)`: retorna la traducción desde el `ResourceBundle` correspondiente.
- Los recursos se cargan desde `src/bundles/Textos_[idioma].properties`.

5.4.2. GestorUsuarios

Gestión de usuarios y autenticación:

- `registrarUsuario(telefono, contrasena)`: agrega una línea a `usuarios.txt` con formato `telefono:contrasena:cliente`.
- `obtenerRol(telefono, contrasena)`: verifica credenciales y retorna "admin", "cliente" o null.
- `existeUsuario(telefono)`: comprueba si el usuario ya está registrado.
- Validación de formato de teléfono: exactamente 9 dígitos.
- Validación de contraseña: mínimo 8 caracteres.

5.4.3. GestorCitas

Gestión de citas y persistencia:

- `leerCitas()`: lee todas las líneas de `citas.txt`.
- `guardarCita(hora, cliente)`: appenda una nueva cita al archivo.
- `eliminarCita(linea)`: reescribe `citas.txt` omitiendo la línea especificada.
- Formato de almacenamiento: `hora - nombreCliente`.

5.5. Diálogos Modales

5.5.1. DialogoErrorLogin

Ventana modal que aparece cuando falla la autenticación:

- Encabezado amarillo con el mensaje de error.
- Botones: **Reintentar** (vuelve al login) y **¿Olvidó su contraseña?** (va a recuperar).

5.5.2. DialogoAnadirCita

Ya descrito anteriormente como parte del flujo de administrador.

5.6. Capturas de pantalla

A continuación se incluyen las pantallas principales implementadas:

Pantalla 1: Inicio de Sesión

Formulario de autenticación con campos de teléfono y contraseña. Botón de cambio de idioma en esquina superior derecha. Botones para iniciar sesión, registrarse y recuperar contraseña.

Figura 5.1: Pantalla de login

Pantalla 2: Registro de Usuario

Formulario con campos de teléfono (9 dígitos), contraseña (8+ caracteres), confirmación de contraseña. Validación integrada. Botones de registrarse y cancelar.

Figura 5.2: Formulario de registro

Pantalla 3: Recuperación de Contraseña

Interfaz simple con búsqueda por teléfono. Muestra la contraseña del usuario encontrado o error si no existe. Botones de búsqueda y retorno.

Figura 5.3: Pantalla de recuperación

Pantalla 4: Menú del Cliente

Dos botones principales: “Pedir cita” (azul/oscura) y “Anular cita” (amarillo).
Opción de cambio de idioma y cierre de sesión. Navegación bimodal.

Figura 5.4: Menú principal del cliente

Pantalla 5: Reserva de Cita (Cliente)

Formulario con desplegables para día, hora y tipo de servicio. Campo de confirmación. Botones de guardar y cancelar. Persiste en `citas.txt`.

Figura 5.5: Pantalla de reserva de cita

Pantalla 6: Anulación de Cita (Cliente)

Lista de citas del cliente autenticado. Botón de anulación para eliminar citas. Botón de retorno al menú. Actualiza `citas.txt` dinámicamente.

Figura 5.6: Pantalla de anulación de cita

Pantalla 7: Panel de Administración Principal

Cuatro botones de navegación en fila: Inicio, Agenda, Clientes, Inventario. Área central que cambia dinámicamente según la selección. Opción de cierre de sesión.

Figura 5.7: Panel de administración

Pantalla 8: Agenda del Administrador

Lista completa de citas del sistema. Botones para añadir cita (abre diálogo modal) y anular cita seleccionada. Gestión centralizada de todas las reservas.

Figura 5.8: Panel de gestión de agenda

Pantalla 9: Clientes y Fichas

Grid de clientes en columnas de 3. Cada cliente es un botón con su nombre. Al seleccionar, abre formulario de ficha con campos de alergias, historial y teléfono.
Guardado en `fichas_clientes.txt`.

Figura 5.9: Panel de gestión de clientes

Pantalla 10: Inventario (Productos)

Grid de productos con stock visible. Botones para aumentar/disminuir cantidad.
Botón de eliminación. Gestión dinámica del inventario.

Figura 5.10: Panel de inventario

Capítulo 6

Diferencias respecto al prototipo

6.1. Cambios de diseño justificados

La implementación final presenta varias diferencias respecto al prototipo del proyecto presentado en la primera entrega. A continuación se detallan y justifican:

6.1.1. Eliminación de iconografía visual (emojis)

En el prototipo se incluían emojis y símbolos gráficos como “” para idioma, “” para el logo, “” para avatares, etc. En la implementación final, estos se han eliminado porque:

- Garantizan compatibilidad universal con cualquier entorno Java.
- Evitan problemas de codificación en sistemas heterogéneos.
- Mantienen la interfaz limpia y profesional.
- Se utilizan componentes de texto plano en su lugar.

6.1.2. Estructura de datos simplificada

El prototipo planteaba campos complejos; la implementación utiliza archivos de texto plano:

- `usuarios.txt`: formato `telefono:contrasena:rol`

- `citas.txt`: formato hora - cliente
- `fichas_clientes.txt`: formato simple con datos de alergias, historial y contacto.

Esta decisión simplifica la persistencia y es apropiada para una práctica de interfaz gráfica sin base de datos.

6.1.3. Validaciones más estrictas

Se han implementado validaciones más robustas que en el prototipo:

- Validación de teléfono: exactamente 9 dígitos (expresión regular).
- Validación de contraseña: mínimo 8 caracteres.
- Prevención de duplicados al registrarse.

6.1.4. Gestión de pantallas revisada

En lugar de un cambio de vista simplista, se utiliza:

- Método `cambiarVista()` en `VentanaPrincipal` que revalida contenedores.
- Método `cambiarVistaInterna()` en `PanelAdminPrincipal` para navegación dentro de la sección admin.
- Transiciones explícitas que garantizan actualización de la interfaz.

6.1.5. Internacionalización completa

El prototipo no incluía i18n. La implementación:

- Incluye 3 idiomas (español, inglés, francés).
- Utiliza `ResourceBundle` desde los archivos `src/bundles/Textos_[idioma].properties`.
- Permite cambio dinámico de idioma en cualquier momento.

6.1.6. Componentes más modulares

Cada pantalla es una clase `JPanel` independiente, permitiendo:

- Reutilización y mantenibilidad.
- Encapsulación de lógica de cada vista.
- Facilidad para agregar nuevas pantallas.

6.2. Elementos conservados del prototipo

- Estructura de dos roles (administrador y cliente).
- Funcionalidades principales: agenda, clientes, inventario.
- Flujos de autenticación con registro y recuperación.
- Diseño de interfaz intuitivo y accesible.
- Nombre de la aplicación: “Laura Estilistas”.

Capítulo 7

Conclusiones y Reflexión

7.1. Logros alcanzados

La aplicación implementa la mayoría de los requisitos de la práctica final con éxito:

- **Interfaz gráfica:** Construida con Java Swing, estructurada en 13 pantallas/componentes modulares.
- **Arquitectura:** Separación clara entre lógica de negocio (Gestores) e interfaz (Paneles y Diálogos).
- **Internacionalización:** Soporte de 3 idiomas con cambio dinámico.
- **Persistencia:** Sistema de archivos de texto para usuarios, citas y fichas de clientes.
- **Validaciones:** Implementadas en registros, autenticación y entrada de datos.
- **Conceptos aplicados:** BorderLayout, GridLayout, GridBagLayout, FlowLayout, ActionListener, expresiones lambda, ResourceBundle.

7.2. Aspectos mejorables

De cara a futuras iteraciones, se podría:

- Migrar a una base de datos relacional (SQLite, MySQL) en lugar de archivos de texto.

- Implementar búsqueda y filtrado avanzado de citas y clientes.
- Agregar reportes y estadísticas de uso.
- Mejorar el diseño visual con temas personalizados.
- Implementar autenticación más segura (hashing de contraseñas).
- Agregar sincronización y multi-usuario en tiempo real.

7.3. Conclusión final

Esta práctica ha permitido afianzar los conocimientos de Java Swing, diseño de interfaces gráficas, gestión de eventos y arquitectura de aplicaciones con separación de responsabilidades. La implementación respeta los conceptos enseñados en la asignatura “Sistemas Interactivos” y demuestra capacidad para traducir un prototipo en papel a un aplicativo funcional.

Apéndice A

Anexos

A.1. Comandos útiles

- Compilar: `javac -d bin src/*.java`
- Ejecutar: `java -cp bin VentanaPrincipal`
- Cambiar idioma: usar el botón de idioma en pantalla de login.